中国地质大学（武汉）2024年新生赛（同步赛）个人题解

（写给自己看的，不喜勿喷）

暂时只做出来了ABCDFGHJKL

[中国地质大学（武汉）2024年新生赛（同步赛）]

A题-数组

[A-数组_中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

在计算机科学中，数组是一种常用的数据结构。数组的一个重要特性是其下标（索引）通常从 0 开始。对于一个长度为 n 的数组，其元素下标范围是从 0 到 n−1。

作为刚入门 C++ 编程的新生，lzt 需要完成一个简单的任务来巩固对数组下标的理解。具体任务如下：

给定一个整数 n，请输出 n−1。这个操作模拟了获取数组最后一个元素的下标。

输入描述:

共一行。第一行包含一个整数 n(1≤n≤100)，表示数组长度。

输出描述:

输出一个整数，表示这个数组最后一个元素的下标。

示例1

输入

```
43
```

输出

```
42
```

题解

签到题

```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin>>n;
    cout<<n-1<<endl;
    return 0;
}
```

B题-借阅图书

B-借阅图书_中国地质大学（武汉）2024年新生赛（同步赛）

题目描述

图书馆新进了一批特别火的图书，吸引了一大批读者借用。lzt作为图书管理员，想要维护借书秩序。于是他先把这n本书依次标号为1,2,...,n，然后找到一个超级大箱子，把这些书按顺序放在箱子里面(序号为n的图书在箱子顶部)，之后他打算对这些书进行图书管理，并向读者公布借用规则，规则如下：

1. 开放预约制。若此时读者没有正在借阅的图书，则该读者可以预约d天后进行借阅，但图书馆**无法保证**读者能在d天后能借到书。并且，该读者只能等到d天之后进行下一次预约。
2. 在每一天的一开始(即在当天进行操作之前)，图书馆会从箱子顶端依次取出图书来试图满足之前预约过当天借阅的读者的需求。越早预约的读者越先被分配到，并且给分配到图书的读者的状态变为"正在借阅"。若在此过程中lzt发现箱子里的图书空了，则之后预约在当天的读者将不会分配到图书，这些预约也会**作废**。
3. 可以随借随取。只要箱子里的图书不空，并且这个读者此时没有预约该天之后的图书，同时也没有正在借阅的图书，则请求立即借阅的读者将立即取得箱子顶部的图书。
4. 读者归还图书后，lzt会把该图书放到箱子顶端。
5. 每个读者同一时间最多只能借阅一本书。

此外，每个读者都有一个唯一的**读者ID**。图书馆需要支持查询特定读者当前借阅的图书编号。

现在，lzt 需要你维护图书馆的借阅过程。

输入描述:

第一行包含两个整数 n,m(1≤n,m≤1000)，分别表示图书的数量和操作的数量。

接下来 m 行，每行描述一个操作，操作的类型有四种：

1. "t RESERVE id d"：表示在第t天读者 id 预约在当前操作所在的第 d(1≤d≤1000)天后借书。如果读者 id 已经有未归还的图书，或者之前已经进行过预约，则此次预约无效，忽略该操作。
2. "t BORROW id"：表示在第t天读者 id 请求立即借书。
3. "t RETURN id"：表示在第t天读者 id 归还所借的图书。如果读者 id 没有借阅任何图书，则忽略该操作。
4. "t QUERY id"：表示在第t天查询读者 id 当前借阅的图书编号。

注意：

1. 操作按照时间顺序依次进行。
2. 多个操作可以发生在同一天。输入中每个操作前会指定其执行的天数。
3. 天数从 1 开始(但第一天不一定有操作)，且操作按非递减的天数顺序给出。
4. 每个读者的 id(1≤id≤1000) 是一个唯一的正整数。
5. t(1≤t≤1000) 是一个整数，表示该操作发生时所在天数。

输出描述:

1. 对于每一个 "RESERVE" 操作，输出一行一个整数。若操作成功，则输出1，否则输出0 .
2. 对于每一个 "BORROW" 操作，输出一行一个整数，表示被借出的图书编号。如果借书请求无效或箱子为空，则输出 0。
3. 对于每一个 "RETURN" 操作，输出一行一个整数，表示归还的图书编号。如果归还操作无效（该用户没有正在借阅的书），则输出 0。
4. 对于每一个 "QUERY" 操作，输出一行一个整数，表示读者当前借阅的图书编号。如果读者未借阅任何图书，则输出 0。

示例1

输入

```
2 10
1 RESERVE 1 2
2 BORROW 1
2 RESERVE 1 3
2 BORROW 2
2 RESERVE 3 4
4 BORROW 3
5 RETURN 1
5 QUERY 2
6 QUERY 3
6 QUERY 1
```

输出

```
1
0
0
2
1
0
1
2
1
0
```

题解

模拟题，要注意的是输入中的天数不一定是连续的（比如示例中没有第三天），以及预约操作给的day表示的是在当天的day天后预约（即t+day）

```
class Library:
    def __init__(self, n):
        self.stack = list(range(1,n+1))
        self.borrowed = {}
        self.reservations = {}

    def reserve(self, reader_id, d):
```

```
        if reader_id in self.borrowed or reader_id in self.reservations:
            return 0
        self.reservations[reader_id] = d
        return 1

    def borrow(self, reader_id):
        if reader_id in self.borrowed or reader_id in self.reservations:
            return 0
        if self.stack:
            book_id = self.stack.pop()
            self.borrowed[reader_id] = book_id
            return book_id
        return 0

    def return_(self, reader_id):
        if reader_id not in self.borrowed:
            return 0
        book_id = self.borrowed.pop(reader_id)
        self.stack.append(book_id)
        return book_id

    def query(self, reader_id):
        if reader_id in self.borrowed:
            return self.borrowed[reader_id]
        return 0

    def handle_reserve(self, day):
        to_remove = []

        for reader_id in self.reservations:
            d = self.reservations[reader_id]
            if d == day:
                if self.stack:
                    book_id = self.stack.pop()
                    self.borrowed[reader_id] = book_id
                to_remove.append(reader_id)
        for reader_id in to_remove:
            del self.reservations[reader_id]

n, m = map(int, input().split())
library = Library(n)
pret = 0
for _ in range(m):
    command = input().strip().split()
    t = int(command[0])
    if t!= pret:#处理天数不是连续的情况
        for nt in range(pret+1,t+1):
            library.handle_reserve(nt)
        pret = t
    operation = command[1]
    if operation == "RESERVE":
        reader_id = int(command[2])
        d = int(command[3])
        print(library.reserve(reader_id, t + d))
    elif operation == "BORROW":
        reader_id = int(command[2])
```

```
        print(library.borrow(reader_id))
    elif operation == "RETURN":
        reader_id = int(command[2])
        print(library.return_(reader_id))
    elif operation == "QUERY":
        reader_id = int(command[2])
        print(library.query(reader_id))
```

C题-文神的序列

[C-文神的序列_中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

文神有一个长度为 n 的序列 a，所有元素均为正整数，她可以对序列中的元素进行操作。在每次操作中，她都会选择两个数字 a_i和 a_j ($i \neq j$ 且$1 \leqslant i, j \leqslant n$)，将 a_i改为 a_i& a_j ('&'代表按位与运算)。文神想知道经过至多 n 次操作后，是否能将序列中的所有数字改为 0。

例如：3&5 即 0000 0011&0000 0101=0000 00010000因此 3&5的值为 1。

输入描述:

第一行输入一个整数n(1≤n≤3000)n(1≤n≤3000)，代表序列中元素的个数。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$($1 \leqslant a_i \leqslant 2^{30} - 1$) - 序列a 。

输出描述:

若经过至多 n 次操作后，能将序列中的所有数字改为 0，输出"Yes"；否则输出"No"。

注意输出的是引号里面的内容，输出不包含引号。

示例1

输入

```
3
1 2 5
```

输出

```
Yes
```

题解

比赛的时候凭直觉写了以下代码

```
#include <iostream>
using namespace std;
int main(){
    int n;
```

```
    cin>>n;
    int total,temp;
    cin>>total;
    for(int i=0;i<n-1;i++){
        cin>>temp;
        total = total&temp;
    }
    if(total==0){
        cout<<"Yes"<<endl;
    }
    else{
        cout<<"No"<<endl;
    }
}
```

赛后思考了一下，题目希望我们将整个a序列在n次操作以内所有元素变为0。

注意到0&n = 0,所以只要序列中有0或者序列中存在两个数a,b a&b=0，那么n次操作内一定能将a序列所有元素变为0。

更好理解的代码如下（参考了其他人的ac）

```
#include <iostream>
using namespace std;
long long a[10000];
int main()
{
    int n;
    cin>>n;
    for (int i = 0; i < n; i++) {
        cin>>a[i];
        if (a[i] == 0) {
            cout<<"Yes"<<endl;
            return 0;
        }
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i; j < n; j++) {
            if ((a[i] & a[j]) == 0) {
                cout<<"Yes"<<endl;
                return 0;
            }
        }
    }
   cout<<"No"<<endl;
}
```

自己写的也可以在优化一下

```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin>>n;
```

```
    int total,temp;
    cin>>total;
    if (n==1&&total==0){
        cout<<"Yes"<<endl;
        return 0;
    }
    for(int i=0;i<n-1;i++){
        cin>>temp;
        total = total&temp;
        if(total==0){
            cout<<"Yes"<<endl;
            return 0;
        }
    }
    cout<<"No"<<endl;
}
```

D题-低谷(easy)

[D-低谷(easy) 中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

这是本题的简单版本。在这个版本中，n的数据范围更小。同时，本题困难版本仅在同步赛中出现，以替代校内赛的交互题。

有一个长度为n的数组，现在可以对其进行kkk次操作，每次操作过程如下：

- 对于数组a,选定一个下标$i(1 \leqslant i \leqslant n)$，令$a_i$的值减一。

现定义低谷：

- 对于数组a，若对于下标$i(2 \leqslant i \leqslant n-1)$，使得$a_i < a_{i-1}$且$a_i < a_{i1}$，则称i为数组a的一个低谷。

lzt想知道k次操作后，该数组的低谷最多有多少个。

输入描述:

共两行。第一行包含两个整数$n, k(1 \leqslant n \leqslant 3000, 1 \leqslant k \leqslant 10^9)$，分别表示数组长度n和操作次数k。

第二行包含n个整数$a_1, a_2, \ldots\ldots, a_n(1 \leqslant a_i \leqslant 10^9)$，表示数组aaa。

输出描述:

输出一行一个整数ans，表示数组的低谷最大个数。

示例1

输入

```
7 3
2 2 3 4 6 2 3
```

输出

```
3
```

题解

可以用动态规划来解。

先讲讲我一开始的思路

定义： dp[i][j]表示在数组的前i个元素中，进行j次操作后，能够形成的最多低谷数量。

状态转移:

对于每个位置 i，我们有两种选择：

不操作当前位置 i，则 dp[i][j] = dp[i-1][j]。

操作当前位置 i，减少 a[i] 的值，使得 a[i] 成为一个低谷。

转移方程：

```
dp[i][j] = max(dp[i-1][j], dp[i-1][j-cost(i)])
```

cost(i)为将a[i]变成低谷的操作次数

代码如下

```
n, k = map(int, input().split())
a = list(map(int, input().split()))
a.insert(0, 0)
dp = [[0] * (k + 1) for _ in range(n + 1)]
for i in range(2, n):
    for j in range(k + 1):
        dp[i][j] = dp[i-1][j]
        if j >= a[i] - min(a[i-1], a[i+1]) + 1:
            dp[i][j] = max(dp[i][j], dp[i-1][j - (a[i] - min(a[i-1], a[i+1]) +
1)] + 1)
print(dp[n-1][k])
```

但是这段代码有一个致命的问题，其空间复杂度和时间复杂度为O(n*k)，而k的取值范围是 $1 \leqslant k \leqslant 10^9$，百分百爆掉。

不难发现，如果只在意被操作的元素的个数的时候，最多有n个元素被操作。于是可以优化dp[i][j]定义如下

定义： dp[i][j]表示在数组的前i个元素中，形成j个低谷所需要的最小操作次数

状态转移同上

转移方程：

```
dp[i][j]=min(dp[i-1][j],dp[i-2][j-1]+max(0ll,max(a[i]-a[i-1]+1,a[i]-a[i+1]+1)));
```

优化之后只需要开大小为n^2的dp数组，空间复杂度和时间复杂度为O(n^2),远小于O(n*k)

代码如下

```
#include <iostream>
#include <cstring>
#include <vector>
using namespace std;
typedef long long ll;
const ll MAX=0x3f3f3f3f3f3f3f3fll;
inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

int main(){
    ll n,k,a[200005];
    ll dp[3001][3001];
    n = read();
    k = read();
    for (int i=1;i<=n;i++){
        a[i] = read();
        dp[0][i]=MAX;
    }
    for (int i=1;i<=n;i++){
       for (int j=1;j<=n;j++){
           dp[i][j]=MAX;
       }
    }
    for (int i=2;i<n;i++){
        for (int j=n;j>=1;j--){
            dp[i][j]=min(dp[i-1][j],dp[i-2][j-1]+max(0ll,max(a[i]-a[i-1]+1,a[i]-
a[i+1]+1)));
        }
    }
    ll Max = 0;
    //dp中最后一个小于k的即为所求解
    for (int i=n;i>0;i--){
        if (dp[n-1][i]<=k) {
            Max = i;
            break;
        }
    }
    cout<<Max<<endl;
    return 0;
```

```
}
```

F题-猫猫虫的困境II

[F-猫猫虫困境II_中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

去年猫猫虫全部逃出第一个困境出口后，猫猫虫们又来到了第二个困境。猫猫虫们发现自己都在x轴上，随后发现有k个传送门也分布在x轴上，且每个传送门都分别与左侧和右侧最近的一个传送门连接（如果存在），且每次需要一个单位的时间进行传送。困境出口只有1个，即横坐标g处。猫猫虫、传送门和出口都在横坐标为[1,n]之间。p个猫猫虫散布在x轴的不同位置，它们需要从困境出口逃离。每次向左移动一格或向右移动一格都需要一个单位的时间。并且，每次猫猫虫到达传送门时，可以选择无视传送门、向左传送至左侧最近的传送门、向右传送至右侧最近的传送门三种中的一种操作。lzt想知道每只猫猫虫到达困境出口，即到达横坐标为g的点，需要的最少时间。

输入描述:

第一行输入四个整数$n, p, k, g(1 \leqslant g \leqslant n \leqslant 10^9, 1 \leqslant p, k \leqslant 10^5)$，分别表示x轴长度、猫猫虫数量、传送门数量、困境出口横坐标。

第二行输入p个整数$a_1, a_2, ..., a_p$，其中第$i(1 \leqslant i \leqslant p)$个整数表示第i个猫猫虫的横坐标$a_i(1 \leqslant a_i \leqslant n)$,$a_i$互不相同。

第三行输入k个整数$b_1, b_2, ..., b_k$，其中第$i(1 \leqslant i \leqslant k)$个整数表示第i个传送门的横坐标$b_i(1 \leqslant b_i \leqslant n)$,$b_i$互不相同。

输出描述:

共一行。第一行输出p个整数$t_1, t_2, ..., t_p$，其中第$i(1 \leqslant i \leqslant p)$个整数表示第i个猫猫虫到达困境出口最少经过的时间单位。

示例1

输入

```
10 2 2 7
2 8
4 7
```

输出

```
3 1
```

题解

不妨假设在-INF,INF处有两个传送门使得边界无需特判

对于每只猫猫虫，其最小消耗时间为以下五种中的最小值：

1.直接到g,
2.一开始向左到传送门，一直传送到g左边的传送门，向右到g点,
3.一开始向左到传送门，一直传送到g右边的传送门，向左到g点,
4.一开始向右到传送门，一直传送到g左边的传送门，向右到g点,
5.一开始向右到传送门，一直传送到g右边的传送门，向左到g点,

```
#include <iostream>
#include <cstring>
#include <vector>
#include<algorithm>

using namespace std;
typedef long long ll;

const ll MAX=0x3f3f3f3f3f3f3f3fll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

int main(){
    ll n=read(),p=read(),k=read(),g=read();
    vector<ll> a(p+1,0);
    for(ll i=1;i<=p;i++){
        a[i] = read();
    }
    vector<ll> b(k+2,0);
    for(ll i=1;i<=k;i++){
        b[i] = read();
    }
    b[0] = -MAX;
    b[k+1] = MAX;//设置一个远大于n的数使边界不需要特判
    sort(b.begin(),b.end());
    /*对于每只猫猫虫，其最小消耗时间为以下五种中的最小值：直接到g,
    一开始向左/右到传送门，向左/右到g*/
    auto r = lower_bound(b.begin(),b.end(),g);
    auto l = r - 1;
    for(ll i=1;i<=p;i++){
        auto xr = lower_bound(b.begin(),b.end(),a[i]);
        auto xl = xr - 1;

        cout<<min({abs(g-a[i]),
            abs(a[i] - *xl) + abs(xl - l) + abs(g- *l),
              abs(a[i] - *xl) + abs(xl - r) + abs(g- *r),
              abs(a[i] - *xr) + abs(xr - r) + abs(g- *r),
              abs(a[i] - *xr) + abs(xr - l) + abs(g- *l)})<<' ';

    }
}
```

不太理解边界情况的可以先看下面的代码

```
#include <iostream>
#include <cstring>
#include <vector>
#include<algorithm>

using namespace std;
typedef long long ll;

const ll MAX=0x3f3f3f3f3f3f3f3fll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

int main(){
    ll n=read(),p=read(),k=read(),g=read();
    vector<ll> a(p+1,0);
    for(ll i=1;i<=p;i++){
        a[i] = read();
    }
    vector<ll> b(k+2,0);
    for(ll i=1;i<=k;i++){
        b[i] = read();
    }
    b[0] = -MAX;
    b[k+1] = MAX;
    sort(b.begin(),b.end());
    /*对于每只猫猫虫，其最小消耗时间为以下五种中的最小值：直接到g，
    一开始向左/右到传送门，向左/右到g*/
    auto r = lower_bound(b.begin(),b.end(),g);
    auto l = r - 1;
    for(ll i=1;i<=p;i++){
        auto xr = lower_bound(b.begin(),b.end(),a[i]);
        auto xl = xr - 1;
        if (a[i]<b[1]){
            cout<<min({abs(g-a[i]),
                       abs(a[i] - *xr) + abs(xr - r) + abs(g- *r),
                  abs(a[i] - *xr) + abs(xr - l) + abs(g- *l)})<<' ';
        }
        else if(a[i]>b[k]){
            cout<<min({abs(g-a[i]),
                abs(a[i] - *xl) + abs(xl - l) + abs(g- *l),
                  abs(a[i] - *xl) + abs(xl - r) + abs(g- *r)})<<' ';
        }
        else{
            cout<<min({abs(g-a[i]),
                  abs(a[i] - *xl) + abs(xl - l) + abs(g- *l),
                  abs(a[i] - *xl) + abs(xl - r) + abs(g- *r),
                  abs(a[i] - *xr) + abs(xr - r) + abs(g- *r),
```

```
                    abs(a[i] - *xr) + abs(xr - l) + abs(g- *l)})<<' ';
        }
    }
}
```

G题-奥运会

[G-奥运会_中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

奥运会，可以分为夏季奥运会和冬季奥运会，是全球范围内规模最大、最具影响力的体育盛会。夏季和冬季奥运会均为每4年举办一届。其中，最近的一场夏季奥运会为在2024年举办的巴黎奥运会，最近的一场冬季奥运会为在2022年举办的北京冬季奥运会。lzt想知道，在之后的时间里，若奥运会均正常举办，即夏季和冬季奥运会每四年正常举办一届，在L年初到R年末，共举办了几场奥运会？（夏季奥运会和冬季奥运会各单独算一场）

输入描述:

```
输入共一行。第一行输入两个整数L,R(2025≤L≤R≤9999)，含义同上述题面。
```

输出描述:

```
输出共一行。第一行输出一个整数x，表示在L年初到R年末举办的奥运会的场数。
```

示例1

输入

```
2030 2050
```

输出

```
11
```

题解

直接看代码吧，注意L,R为偶数年份的处理

```
#include <iostream>
using namespace std;
int main(){
    int l,r;
    cin>>l>>r;
    if (r%2==0){
        r+=1;
```

```
    }
    if(l%2==0){
        l-=1;
    }
    cout<<(r-l)/2<<endl;
    return 0;
}
```

H题-文神的比赛

[H-文神的比赛_中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

文神主办了一场ICPC赛制的竞赛，共有 n 名选手参与，每个人通过的题目情况用一个字符串表示。字符串中的每个字符代表通过的题目编号，字符由大写字母A到Z构成，字符串的长度不固定且不超过26，每种字符在每个字符串中只出现一次。
每个字符串中的字符表示该选手通过的题目。例如，"ACD" 表示该选手通过了 A 题、C 题和 D 题。

选手的排名规则如下：

1. 通过题目数较多的选手排名靠前。
2. 如果通过的题目数相同，则选手的排名按罚时(保证不存在罚时相同的不同选手)决定，罚时小的排名靠前。

好奇的文神想知道，在只告诉你每个人通过题目的数量，不告诉你每个人罚时的情况下，最终排名会出现多少种情况。
文神太懒了不想自己写代码，请你帮助文神，输出可能的不同排名情况数。结果对 10^97取模。

输入描述:

第一行输入一个整数 n ($1 \le n \le 10^5$)，表示参赛者人数。

接下来n行，每行包含一个字符串，表示每个人通过的题目编号，字符串的长度不超过26，字符串仅包含大写字母 'A' 到 'Z'，每种字符在每个字符串中最多只出现一次。

输出描述:

输出一个整数，表示可能的不同排名情况数，结果对 10^97取模。

示例1

输入

```
4
ABC
ABCD
ABCD
DBAE
```

输出

```
6
```

题解

只有解题数量相同的选手才需要考虑罚时不同造成的不同情况。

因此先统计各解题数量的选手都要几个。

对于相同的解题数量的t个选手，罚时不同的情况有A_t^t（即$t!$）种情况。将所有解题数量的情况相乘即是答案。

注意对阶乘进行记忆优化降低时间复杂度

```
#include <iostream>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
typedef long long ll;

const ll MOD = 1000000007;
vector<ll> factorialMemo(2,1);


ll factorial(ll n) {
    if (n<(ll)factorialMemo.size()) {
        return factorialMemo[n];
    }
    ll t =factorialMemo[factorialMemo.size()-1];
    for (ll i=factorialMemo.size();i<=n+1;i++){
        t = (t*(i))%MOD;
        factorialMemo.push_back(t);
    }
    return factorialMemo[n];
}


int main(){
    ll nums[27];
    memset(nums, 0, sizeof nums);
    ll n;
    cin>>n;
    string s;
    for(ll i=0ll;i<n;i++){
        cin>>s;
        //cout<<s.length()<<endl;
        nums[s.length()] += 1;
    }
    ll total=1,num;
    for(ll i=1;i<=26;i++){
        if (nums[i]!=0){
            num = nums[i];
            //cout<<i<<' '<<nums[i]<<endl;
```

```
            total = (total * factorial(num))%MOD;
        }
    }
    printf("%lld\n",total);
    return 0;
}
```

J题-星穹列车

[J-星穹列车_中国地质大学（武汉）2024年新生赛（同步赛）]

星穹列车，不许发车。嗷呜~~~

题目描述

在二维平面上有一班星穹列车从(x0,y0)出发，沿着方向向量$\overrightarrow{v}$=(1,0)（即x轴正方向）行驶，一直驶向银河彼岸（即可看作目的地无穷远）。其中，有k个宇宙风暴，这些宇宙风暴在该平面上呈现为圆心坐标为(x_d,y_d)，半径为r_d的圆形。lzt想知道，镜流在列车行驶过程中，需要经过的宇宙风暴长度。

输入描述:

第一行输入三个整数$x_0, y_0, k(-10^9 \leqslant x_0, y_0 \leqslant 10^9, 1 \leqslant k \leqslant 10^5)$，分别表示星穹列车的初始横坐标，纵坐标，以及危险区域个数k。

后面k行，第i行输入三个整数$x_{d_i}, y_{d_i}, r_{d_i}(-10^9 \leqslant x_{d_i}, y_{d_i} \leqslant 10^9, 1 \leqslant r_{d_i} \leqslant 10^9)$，表示第i个宇宙风暴的圆心横坐标xdi，纵坐标ydi，半径。

输出描述:

共一行。第一行输出一个实数ans，表示在列车行驶过程中，需要经过的宇宙风暴长度。

设你的答案为x，标准答案为y，只有当x,y满足时$\frac{|x-y|}{min(1,y)} \leqslant 10^{-5}$，也就是说，只有当x关于y的绝对误差或相对误差小于等于时，你的答案才会被视为正确。

示例1

输入

```
0 0 4
10 0 5
20 0 10
30 0 15
-2 -3 5
```

输出

```
42.00000000000000000000
```

示例2

输入

```
0 1 2
0 0 1
1 5 3
```

输出

```
0.00000000000000000000
```

题解

很经典的区间合并（不是这道题的唯一解法）。

首先求出每个圆与y=x0相交形成的弦（未相交忽略），用左右端点构造一个区间，在进行区间合并就行了。

合并区间可以看看力扣上的题解[56. 合并区间 - 力扣（LeetCode）](#)

要注意区间不能在x0左边（有笨比因为这个点卡在了J题上两个小时）

```
import math

def merge(intervals):
        intervals.sort(key=lambda x: x[0])
        new_list = list()
        for i in intervals:
            if not new_list:
                new_list.append(i)
                continue
            if new_list[-1][1] >= i[0]:
                x = max(new_list[-1][1], i[1])
                new_list[-1][1] = max(new_list[-1][1], i[1])
            else:
                new_list.append(i)
        return new_list


x0, y0, k = map(int, input().split())
intervals = []

for _ in range(k):
    x, y, r = map(float, input().split())
    if r * r - (y - y0) ** 2 > 0:
        t = [max(x0, x - math.sqrt(r * r - (y - y0)** 2)),
             max(x0,x + math.sqrt(r * r - (y - y0)** 2))]
        intervals.append(t)

if not intervals:
    print("0.00000000000000000000")
else:
    merged_intervals = merge(intervals)
```

```
    ans = sum([interval[1] - interval[0] for interval in merged_intervals])
    print(f"{ans:.20f}")
```

K-冰红茶

[K-冰红茶_中国地质大学（武汉）2024年新生赛（同步赛）]

see you again~

题目描述

给定$n \times m$的网格（n行m列），再给定k瓶冰红茶，每瓶冰红茶有它自己的价格，你需要将**所有**冰红茶放进这$n \times m$的网格中，lzt会从每一行中都等概率随机拿取一瓶冰红茶（若该行没有冰红茶，则忽略该行），并且支付这瓶冰红茶的钱，cxh作为老板，想知道应该怎样放置可以使得lzt付的钱数期望最大(不要求每一行都必须有冰红茶，同时一个方格最多只能放置一瓶冰红茶)

输入描述:

共两行。

第一行输入三个整数n,m,k($1 \leqslant m, n \leqslant 1000, 1 \leqslant k \leqslant n \times m$)，含义如上述题面所示。

第二行输入k个整数$p_1, p_2, \ldots\ldots, p_n$，其中第i个整数表示第i瓶冰红茶的价格$p_i(1 \leqslant p_i \leqslant 10^9)$。

输出描述:

输出题意所求n行m列的网格v。

具体地，输出n行，其中第i行输出m个整数，每个整数之间用空格隔开。若第i行第j列不放置冰红茶，则$v_{i,j} = 0$；否则$v_{i,j} = index(1 \leqslant index \leqslant k$,index为整数)，其中index表示冰红茶对应的下标，即：第index瓶冰红茶放置在第i行第j列的方格中。

若有多种解决方案，输出任意一种即可。

示例1

输入

```
2 3 3
1 4 6
```

输出

```
1 2 0
3 0 0
```

题解

考虑到要让期望尽可能大，则需要将大的数将可能单独放一行。

当剩下的放不下时，仍需注意要让大的在一行且数量将可能少。

（对于一行来说，其价格期望就是其放置的冰红茶的价格的平均数，因此大的数尽可能单独一行，如果不能，也应当使其与最接近的数一行且一行中的数量尽可能少）

因此我们先对p排序，在按上述思路填入即可

```
n, m, k = list(map(int, input().split()))
p =  list(map(int, input().split()))
for i in range(k):
    p[i] = [i+1,p[i]]
p.sort(key = lambda x:x[1],reverse=True)

idx = 0
maze = [[0 for i in range(m)] for i in range(n)]
i = 0
can = int((m*n-len(p))/(m-1)) if m!= 1 else min(n-1,len(p))
for i in range(can):
    maze[i][0] = p[idx][0]
    idx += 1
    if idx == len(p):
        break


i = can
j = 0
nidx = len(p) - 1
while nidx>=idx:
    maze[i][j] = p[nidx][0]
    nidx -= 1
    j+=1
    if j%m== 0 and j:
        i += 1
        j = 0

for i in maze[::-1]:
    print(' '.join(map(str,i)))
```

L题-数组交换

[L-数组交换_中国地质大学（武汉）2024年新生赛（同步赛）]

题目描述

给定一个大小为n(n为奇数)的数组a。现在进行$\frac{n-1}{2}$轮操作，每轮操作必须要：

1. 选定一个下标i($1 \leqslant i \leqslant n-1$)，交换$a_i$和$a_{i1}$;
2. 删除数组a最前面的两个元素a_1和a_2，得到一个新的数组 $a' = \{a_1', a_2', \ldots\ldots, a_{n-2}'\} = \{a_3, a_4, \ldots\ldots, a_n\}$，以及新的数组元素大小 $n' = n-2$;
3. 令a=a',n=n'，更新数组a和数组a的大小n。

完成之后，此时数组仅剩下一个元素。lzt需要采取一定策略，使得这个元素最大。求这个元素的值。

输入描述:

第一行输入一个整数n($1 \leqslant n \leqslant 2 \times 10^5$，n为奇数)，表示数组a的元素个数。

第二行输入n个整数$a_i(1 \leqslant a_i \leqslant 10^9)$，表示数组a中的第i个元素。

输出描述:

共一行。第一行输出一个整数x，表示所求元素的值。

示例1

输入

```
7
10 6 2 1 5 3 4
```

输出

```
5
```

考虑x尽可能大，我们应当选择a中最大的数。

但同时a的开头两个数在不断被去除，因此我们应当考虑一开始选择哪个数，使其尽可能大的同时不会在被移动到最后前被去除。

我们知道会进行$\frac{n-1}{2}$轮操作，则$n-\frac{n-1}{2}-1$后的数都能来的即移到最末尾位置

不妨考虑以下情况

输入 1 3 2

不难发现3是所求值

输入 1 2 5 3 1

不难发现5是所求值

所以只需求后$\frac{n-1}{2}1$个数中最大的那个就行了

```
#include <iostream>
using namespace std;
int main(){
    int n,t,can,Max;
    cin>>n;
    can = n-(n-1)/2-1;
    for(int i=0;i<can;i++)cin>>t;
    cin>>Max;
    for(int i=can+1;i<n;i++){
        cin>>t;
        if (t>Max){
            Max = t;
        }
    }
    cout<<Max<<endl;
}
```